

Assignment 3: Improving Your Mini Transformer Text Generator

Objective

In class, we built a **Transformer-based character language model** from scratch. In this assignment, you will **improve, experiment with, and turn that model into a usable text generator tool**.

This assignment focuses on:

- Understanding how Transformers use context
- Controlling text generation behavior
- Practicing real ML workflow (saving/loading models)
- Turning a model into a usable program

Academic Integrity

- Plagiarism is strictly prohibited
 - You may use libraries **only where allowed**
 - All manual implementations must be your own
-

Dataset

<https://www.kaggle.com/datasets/kingburrito666/shakespeare-plays>

Part 1 — Understanding Context Window

Transformers can only see a limited number of past tokens, controlled by `block_size`.

Task

Train the same model using **three different context lengths**:

```
block_size = 32
block_size = 64
block_size = 128
```

Use the same dataset and similar training steps for all three.

Observe

Generate text samples from each model and compare:

- Does longer context improve coherence?
- Does the model remember longer patterns (like speaker names or sentence structure)?

Deliverables

- 3 generated text samples (one for each block size)
- Short explanation (4–5 lines) of differences you observed

Part 2 — Smart Text Generation (Sampling Methods)

Right now, your model generates text with fully random sampling. You will make it **smarter**.

Read about what is temperature in a Gen AI model and how does it work.

1. Add Temperature Sampling

Modify generation:

```
temperature = 0.8
probs = F.softmax(logits / temperature, dim=-1)
```

Test with:

```
temperature =0.5  
temperature =1.
```

Observe

How does text change?

- Lower temperature → ?

Deliverables

- Generated outputs for different temperature
- Short explanation of how these affect creativity vs coherence

Part 3 — Model Saving and Loading

In real ML systems, models must be saved and reused.

Task

Save your trained model:

```
torch.save(model.state_dict(),"model.pt")
```

Reload it later and generate text **without retraining**:

```
model.load_state_dict(torch.load("model.pt"))  
model.eval()
```

Deliverables

- Screenshot or output showing generation from the reloaded model

Part 4 — Build a Text Generator Script

You will turn your model into a small tool.

Task

Create a Python script called:

```
generate.py
```

It should:

1. Load the saved model
2. Take a **prompt** from the user
3. Generate text
4. length should be used to limit output tokens which is very important for scalability stand point of view.

Example usage:

```
python generate.py --prompt"HAMLET:" --length 200
```

Deliverables

- `generate.py` file
- `model.pt` file
- Screenshot of it running from terminal

Submission Format

Submit:

1. Notebook (`.ipynb`) with experiments
2. `generate.py` script
3. Short PDF or markdown report containing:

- Observations from Part 1
 - Sampling comparisons from Part 2
-